

B02 Python Pipeline Transition

10 lessons | Fast Track | Beginner-first engineering bridge |

Purpose

Turn competent analyst Python into small, reproducible pipeline reasoning without duplicating C3 environment or production-ingestion curriculum.

Study loop

Lesson -> follow once -> save independent attempt -> protected review -> close review -> changed retry -> evidence -> next route. A video never substitutes for the task.

Current source baseline

Target references: Python 3.14.7, pandas 3.0.5, Requests 2.34.2. Current build-runtime checks were executed separately under Python 3.13.5 / pandas 2.2.3 / Requests 2.32.5; do not confuse build-runtime evidence with target-version certification.

This book is your teacher

Use one lesson at a time. Every lesson keeps the same beginner-first sequence even when the engineering idea becomes more technical. Do not open protected review before you save a genuine attempt. After review, close it and complete the changed retry before mastery.

Daily loop **understand -> picture -> new words -> watch/read if useful -> follow once -> fresh task -> validate -> teach back -> protected review -> changed retry -> evidence**

Contents / Index

Lesson	Title	Pages	Bridge classification
B02-L01	From notebook task to reusable script contract	3	C1 -> script contract deepening
B02-L02	Functions, modules and decomposition for repeatable work	5	C1 -> modularity deepening
B02-L03	Deterministic file discovery and multi-file batches	7	NEW file-batch reasoning
B02-L04	pandas merge and validation contracts for pipelines	9	C1 -> merge contract deepening
B02-L05	CSV, JSON and Excel interfaces with explicit parsing rules	11	MERGED interface contract
B02-L06	HTTP API basics for data acquisition	13	NEW API basics
B02-L07	Error handling, failure classification and containment	15	NEW failure strategy
B02-L08	Logging, run evidence and validation gates	17	NEW run evidence
B02-L09	Recurring batch execution and safe rerun reasoning	19	DEEPER rerun/idempotence
B02-L10	Build and defend a small reproducible batch pipeline	21	SYNTHESIS / C3 entry rehearsal

Fast-track boundary

Virtual environments/dependency pinning, CLI design, packaging, Linux shell/Git and Docker are C3-00 lessons. API auth/pagination/rate limits/retries and incremental state are C3-01 lessons. B02 previews why these matter but does not require mastery.

Protected assessment rule

Gate B/C and protected review material remain sealed until the correct saved-attempt -> review/repair -> explicit reassignment route allows them.

B02-L01 - From notebook task to reusable script contract

Bridge route

What you already know as an analyst: notebook cells can load, clean and summarize data.

What changes in engineering: the run begins from one deliberate entry point and receives explicit paths/config.

Why engineers care: another run, machine or scheduler cannot depend on your forgotten cell state.

Small worked example: `main(input_path, output_path)` coordinates load -> transform -> validate -> publish.

Real engineering version: C3 will add project packaging, dependency environments, CLI design and tests.

1. Why it matters

An analyst notebook can succeed because cells were run in a lucky order. An engineering-style script must succeed from a clean start with explicit inputs, outputs and failure boundaries.

2. What you are learning

Turn a familiar notebook workflow into a reusable script contract with an entry point, explicit paths and no hidden notebook state.

3. Prerequisite

Course 1 PY0/PY9/PY10: you can run a notebook, save outputs and explain reproducibility. No command-line or packaging mastery is assumed.

4. Picture it in your head

Picture a kitchen recipe. A notebook can contain half-used bowls already on the counter. A script contract starts with an empty kitchen: it names ingredients, steps, output and what happens if an ingredient is missing.

5. New words before we use them

script - a Python file intended to run as a program

entry point - the place execution intentionally begins

hidden state - a value/file/object that exists only because something earlier happened outside the declared run

contract - the declared inputs, outputs, rules and failure behavior

6. Watch / visual source

Official Modules + pathlib are authority. No video is required. If you need a visual, use the B02-L03 pathlib companion later.

[Python 3.14.7 Modules](#) | [Python 3.14.7 pathlib](#)

7. Follow with me once

Copy one simple C1 analysis flow into a fresh .py file. Restart Python, run only that file, and record the exact input path, output path and output row count.

```
from pathlib import Path

def main(input_path: Path, output_path: Path):
    raw = load_data(input_path)
    clean = transform(raw)
    validate(clean)
    write_output(clean, output_path)

if __name__ == "__main__":
    main(Path("Practice_Data/sales_2026_01.csv"), Path("outputs/clean_sales.csv"))
```

8. What you should see

A clean process creates the same output without you manually running setup cells first.

B02-L01 - Practice to mastery

9. Why it worked

main() makes the start explicit; paths enter the workflow instead of hiding in notebook state; validation runs before publish.

10. Common beginner mistakes

- Copying every notebook cell unchanged into one giant script.
- Using current working directory assumptions you cannot explain.
- Creating final output before validation finishes.
- Treating if `__name__ == "__main__"` as magic syntax instead of an entry-point boundary.

11. If stuck

Recovery ladder

1) Locate the single input file. 2) Make a tiny load-only main(). 3) Print/record row count. 4) Add one function at a time. Do not add venv/CLI complexity here.

12. Small version alone

Create a fresh script that reads `sales_2026_02.csv` and writes a new CSV to an outputs folder. Input/output paths must be arguments to main(), not scattered constants.

13. Validate

Restart the interpreter and rerun. Compare output row count to source; prove output is new and source bytes are unchanged.

14. Teach it back

In 60-90 seconds explain notebook state vs a clean-start script and name one hidden dependency that could break automation.

15. Protected review + changed retry

Attempt protection

Save the first attempt. Only then open Review B02-L01. Close it and rebuild the same contract using `sales_2026_03.csv` and a different output filename.

16. Evidence / mastery + next route

Ready when

Save script, command/run method, source/output paths, source/output row counts, one failure/recovery note, confidence 1-5. PASS -> B02-L02.

B02-L02 - Functions, modules and decomposition for repeatable work

Bridge route

What you already know as an analyst: functions reduce repeated notebook code.

What changes in engineering: functions also create failure/test boundaries and separate I/O from transformation.

Why engineers care: you can diagnose, replace or test one stage without rewriting the run.

Small worked example: `require_columns(df, required)` raises before bad data reaches merge.

Real engineering version: C3 will turn these into project modules/packages with automated tests.

1. Why it matters

A pipeline becomes hard to test when loading, cleaning, validation and publishing are tangled in one block. Small explicit functions let you isolate failure and reuse only the part that should change.

2. What you are learning

Decompose a workflow into coherent functions with explicit inputs/returns, and understand the module as a reusable file boundary.

3. Prerequisite

B02-L01 mastered; C1 practical Python functions are familiar.

4. Picture it in your head

Imagine four labeled workstations: LOAD, TRANSFORM, VALIDATE, PUBLISH. If a result is wrong, you inspect the relevant station instead of dismantling the whole factory.

5. New words before we use them

function contract - what a function accepts, returns, changes and may raise

module - a Python file that can define reusable code

side effect - a change outside the returned value, such as writing a file

pure-ish transform - a transformation that mostly depends on explicit inputs and returns an output without hidden external changes

6. Watch / visual source

Python Modules is the current authority. A separate functions video is unnecessary because C1 already taught function syntax.

[Python 3.14.7 Modules](#)

7. Follow with me once

Take your L01 script and split it into `load_sales()`, `transform_sales()`, `validate_sales()` and `publish_sales()`. For each, write one sentence: input, output, possible failure.

```
def require_columns(df, required):
    missing = sorted(set(required) - set(df.columns))
    if missing:
        raise ValueError(f"Missing columns: {missing}")

def normalize_channel(series):
    return series.astype("string").str.strip().str.lower()
```

8. What you should see

`main()` becomes short and readable; each function has one coherent responsibility.

B02-L02 - Practice to mastery

9. Why it worked

Decomposition makes dependencies visible. A schema error can fail in validation without being confused with output-writing logic.

10. Common beginner mistakes

- One 100-line function named `process_data`.
- A function secretly reads a global dataframe instead of accepting it.
- Validation function silently edits data to make checks pass.
- Catching every exception inside each function and returning `None`.

11. If stuck

Recovery ladder

Start with only two functions: load and validate. Make them work. Then isolate transformation; publish is last.

12. Small version alone

Add a reusable `require_columns()` function and use it against the missing-column fixture. It must fail with a useful message naming `customer_id`.

13. Validate

Run against good monthly data and the missing-column fixture. Good input passes; bad input fails before transform/publish.

14. Teach it back

Explain why a function can make code easier to operate even when it does not reduce total lines.

15. Protected review + changed retry

Attempt protection

After saved attempt, review L02; then change the required schema by adding `source_file` and prove the validator catches its absence.

16. Evidence / mastery + next route

Ready when

Save module/script, written function contracts, good/bad run output, failure message and changed retry. PASS -> L03.

B02-L03 - Deterministic file discovery and multi-file batches

Bridge route

What you already know as an analyst: `pd.read_csv("file.csv")`.

What changes in engineering: discover a declared batch, sort it, validate every member and keep source lineage.

Why engineers care: missing/extra/changed files can silently alter production results.

Small worked example: `sorted(Path(...).glob("sales_2026_*.csv"))`.

Real engineering version: C3-01 adds landing metadata, historical batches, incremental state and backfill rules.

1. Why it matters

Real recurring data often arrives as a folder of files. Hard-coding January, February and March is fragile: April arrives and your pipeline quietly ignores it.

2. What you are learning

Discover matching files deterministically, sort them, preserve `source_file` lineage, verify schemas and reconcile combined rows to file-level controls.

3. Prerequisite

L01-L02 mastered; you can read one CSV.

4. Picture it in your head

Think of an airport baggage belt. Each bag has a tag showing where it came from. You do not just dump bags into one pile; you list which bags arrived, in a predictable order, and keep the tag after combining them.

5. New words before we use them

file discovery - finding inputs by declared path/pattern rather than manual list

deterministic - same valid inputs/rules produce the same selection/order/result

lineage - evidence of where a row/file came from

batch - a bounded group of records/files processed together

6. Watch / visual source

Book + current pathlib documentation are primary. The previously added Real Python YouTube companion remains recorded in the source audit, but no exact useful segment could be independently reverified, so it is not assigned as required learner media.

[Python 3.14.7 pathlib](#)

7. Follow with me once

Discover the three monthly files, save their ordered names and each row count, add `source_file`, combine them, and reconcile the total to 96 rows.

```
files = sorted(Path("Practice_Data").glob("sales_2026_*.csv"))
parts = []
for path in files:
    part = pd.read_csv(path)
    part["source_file"] = path.name
    parts.append(part)
sales = pd.concat(parts, ignore_index=True)
assert len(sales) == sum(len(p) for p in parts)
```

8. What you should see

Exactly three source filenames in sorted order, 32 rows from each, 96 combined rows, and every combined row has lineage.

B02-L03 - Practice to mastery

9. Why it worked

The pattern is data-driven rather than calendar-code-driven; lineage survives concatenation; reconciliation proves no file was silently lost during combine.

10. Common beginner mistakes

- Using `os.listdir()` order as meaningful.
- Glob pattern accidentally includes outputs or failure fixtures.
- Concatenating before checking schema compatibility.
- Dropping source filename after combining.

11. If stuck

Recovery ladder

Print the matched paths first. Do not concatenate until the list is exactly the intended batch. Then inspect columns for each file.

12. Small version alone

Create a temporary folder containing only two copied monthly files. Point the same discovery function at it; no code changes except root path.

13. Validate

Expected file count and actual file list are saved; combined rows equal sum of component rows; `source_file` counts reproduce file row counts.

14. Teach it back

Explain why sorted discovery + lineage is more reliable than a hard-coded filename list.

15. Protected review + changed retry

Attempt protection

Review after attempt; then add a renamed nonmatching CSV and prove it is excluded by the contract rather than accidentally ingested.

16. Evidence / mastery + next route

Ready when

Save discovered paths, per-file rows/schema, combined rows, lineage count, changed-folder retry. PASS -> L04.

B02-L04 - pandas merge and validation contracts for pipelines

Bridge route

What you already know as an analyst: merge two DataFrames and inspect result.

What changes in engineering: assert master uniqueness, declare many_to_one, measure unmatched keys, reconcile rows/totals.

Why engineers care: pipeline joins need executable contracts, not visual confidence.

Small worked example: current customers.csv deliberately contains two C013 rows.

Real engineering version: C3 uses similar checks as automated data tests and referential-integrity controls.

1. Why it matters

A merge that returns rows is not proof of correctness. A duplicate lookup key can multiply business rows and still create a plausible report.

2. What you are learning

Validate lookup-key uniqueness before merge, use pandas merge cardinality validation, quantify unmatched keys and reconcile business totals.

3. Prerequisite

L03 mastered; C1 PY5 merge and join/cardinality concepts are familiar.

4. Picture it in your head

A customer master should behave like one ID card per customer. If customer C013 has two different ID cards, attaching customer attributes to orders becomes ambiguous; do not choose one silently.

5. New words before we use them

many-to-one - many fact rows may match one master row

validate= - pandas merge parameter that checks expected key cardinality

unmatched key - fact-side key with no master match

critical quality gate - a failed rule that blocks publish

6. Watch / visual source

pandas.merge 3.0.5 is authority. Note its documented null-key behavior differs from SQL and is one reason explicit validation matters.

[pandas 3.0.5 merge](#) | [pandas 3.0.5 User Guide](#)

7. Follow with me once

Try the many_to_one merge with supplied customers.csv. Save the failure. Identify C013. Do not drop one row. Then create an in-memory clean demonstration master only after documenting that the fixture is intentionally ambiguous, and prove row count preservation.

```
if not customers["customer_id"].is_unique:
    dupes = customers.loc[customers["customer_id"].duplicated(False), "customer_id"]
    raise ValueError(f"Duplicate customer keys: {sorted(dupes.unique())}")

enriched = sales.merge(customers, on="customer_id", how="left", validate="many_to_one", indicator=True)
```

8. What you should see

The uncorrected merge is blocked. Your evidence names C013 before publish. A separately controlled demonstration merge preserves 96 sales rows.

B02-L04 - Practice to mastery

9. Why it worked

The pipeline refuses an ambiguous master instead of turning a data-quality problem into duplicate business events.

10. Common beginner mistakes

- `drop_duplicates()` with no winner rule.
- Using `many_to_many` because it makes the error disappear.
- Checking only final row count and not unmatched keys.
- Treating null-to-null pandas matching as SQL-equivalent behavior.

11. If stuck

Recovery ladder

Check `customers.customer_id.value_counts().head()`. Then test `is_unique`. Stop there until you can explain why C013 is ambiguous.

12. Small version alone

Create a tiny fresh orders/master pair with a different duplicated master key. Prove your same validator blocks it without code changes.

13. Validate

Master uniqueness checked before merge; merge cardinality declared; fact rows reconciled; unmatched keys counted; monetary total reconciled where applicable.

14. Teach it back

Explain why duplicate dimension/master keys are not safely repaired by arbitrary deduplication.

15. Protected review + changed retry

Attempt protection

Review after saved attempt; changed retry uses a different duplicate key and one unmatched fact key.

16. Evidence / mastery + next route

Ready when

Save duplicate-key evidence, blocked merge message, controlled demonstration, row/unmatched/total checks. PASS -> L05.

B02-L05 - CSV, JSON and Excel interfaces with explicit parsing rules

Bridge route

What you already know as an analyst: read_csv/read_excel/read_json.

What changes in engineering: declare parsing assumptions and output grain; preserve source; validate serialized output after writing.

Why engineers care: format conversion can lose types, rows or nested meaning.

Small worked example: nested_orders has 5 orders but 10 item rows.

Real engineering version: C3-01 adds defensive parsers, Parquet, landing metadata and schema-drift handling.

1. Why it matters

A file extension does not tell you the business meaning of a row. CSV, nested JSON and Excel each need explicit parsing and output-grain rules.

2. What you are learning

Treat CSV/JSON/Excel as interface contracts: encoding/header/schema/type assumptions, nested-record grain, immutable template/output separation and read-back validation.

3. Prerequisite

L03-L04 mastered; C1 can load files and reshape data.

4. Picture it in your head

Imagine the same shipment arriving in three boxes. One box is flat rows (CSV), one contains boxes inside boxes (JSON), and one is a presentation workbook (Excel). You must know what each layer means before unpacking.

5. New words before we use them

serialization - a way of representing data in a file/message format

nested record - an object containing child objects/arrays

normalization - turning nested structure into a chosen tabular grain

read-back validation - reopening the published file to prove the saved artifact matches intended data

6. Watch / visual source

Optional Corey pandas I/O: 0:56-3:20 CSV; 6:15-12:18 Excel; 12:18-16:59 JSON. After each, use the supplied local files.

[pandas 3.0.5 IO tools](#) | [Python 3.14.7 json](#) | [Corey pandas I/O visual](#)

7. Follow with me once

Load one monthly CSV with explicit required columns. Normalize nested_orders.json to item grain and reconcile 5 parent orders / 10 items. Inspect monthly_report_template.xlsx without overwriting it; write output to a new filename and re-read only if your environment supports Excel engine.

```
payload = json.loads(Path("Practice_Data/nested_orders.json").read_text())
items = pd.json_normalize(
    payload["orders"], record_path="items",
    meta=["order_id", ["customer", "id"], ["customer", "region"]]
)
print("declared grain: one row per order item")
assert len(items) == 10
```

8. What you should see

Your notes distinguish 5 order objects from 10 item rows. Template remains byte-identical. Any created output is separate and validated.

B02-L05 - Practice to mastery

9. Why it worked

The intended grain is chosen before flattening; immutable source/template prevents hidden manual repair; read-back checks the actual published artifact.

10. Common beginner mistakes

- Assuming JSON list length equals desired output rows.
- Overwriting the provided Excel template.
- Using Excel formulas as the hidden source of business truth.
- Relying on automatic dtype inference for critical fields.

11. If stuck

Recovery ladder

Do CSV and JSON first. For Excel, inspect the supplied README sheet; if your local Excel writer engine is unavailable, save CSV evidence and record Excel output as environment-dependent rather than faking it.

12. Small version alone

Create a fresh JSON object with two orders and unequal item counts; normalize to item grain and reconcile parent vs item counts.

13. Validate

Record CSV schema/rows, JSON parent/item counts, template checksum before/after, and published output read-back if executed.

14. Teach it back

Explain why "JSON contains 5 orders" and "normalized table contains 10 rows" can both be correct.

15. Protected review + changed retry

Attempt protection

After review, changed retry removes one optional customer field from a copied JSON payload; keep item reconciliation intact.

16. Evidence / mastery + next route

Ready when

Save parsing contract, normalized sample, counts, template-integrity proof and changed retry. PASS -> L06.

B02-L06 - HTTP API basics for data acquisition

Bridge route

What you already know as an analyst: open/download a file manually.

What changes in engineering: retrieval becomes code with timeout/status/parse validation and an explicit failure path.

Why engineers care: networks fail and a 200 response can still contain unexpected data.

Small worked example: local api_response.json contains 4 region metrics.

Real engineering version: C3-01 teaches authentication, pagination, rate limits, retries/backoff and API contracts in depth.

1. Why it matters

Data does not always arrive as a file. APIs are a common interface, but a successful HTTP connection is not the same as trustworthy data.

2. What you are learning

Understand request/response, GET, status codes, timeout, JSON response parsing and an offline fixture fallback. Authentication, pagination, rate limits and retry strategy are intentionally deferred to C3.

3. Prerequisite

L05 mastered; no previous API experience required.

4. Picture it in your head

An API is like a service counter. You send a request slip to an address; the counter sends a response with a status and a package. You check the status before trusting the package.

5. New words before we use them

HTTP - a protocol for requests and responses on the web

endpoint - a specific API URL for a resource/action

GET - a request method commonly used to retrieve data

status code - numeric response outcome such as 200 or 404

timeout - maximum wait before treating a request as failed

6. Watch / visual source

Requests 2.34.2 Quickstart is authority. Recommended visual: Real Python, Consuming APIs - GET and POST, 01:50-03:02 only (GET -> Response -> .json() -> status code). After watching, immediately run the supplied offline fixture and then the optional local-HTTP GET. The older Corey Requests video remains preserved as backup only. Do not study auth/pagination here as Bridge mastery.

[Requests 2.34.2 Quickstart](#) | [Real Python - Consuming APIs: GET and POST \(01:50-03:02\)](#) | [Corey Requests visual - preserved backup](#)

7. Follow with me once

First parse Practice_Data/api_response.json offline and validate four metrics. Then, if local HTTP is available, serve the Practice_Data folder locally and GET that same file with timeout + raise_for_status().

```
def fetch_json(url, timeout=15):
    response = requests.get(url, timeout=timeout)
    response.raise_for_status()
    return response.json()
```

8. What you should see

Offline and local-HTTP paths produce the same four metric records; network failure does not create/publish a fake empty dataset.

B02-L06 - Practice to mastery

9. Why it worked

Retrieval and validation are separate. timeout bounds waiting; status checking exposes HTTP failure; JSON parsing handles the response body only after transport succeeds.

10. Common beginner mistakes

- No timeout.
- Ignoring status_code and parsing error pages as data.
- Returning [] on every exception and calling the run successful.
- Putting secrets into notebook screenshots/evidence.
- Trying to master pagination/auth here and duplicating C3.

11. If stuck

Recovery ladder

Use offline fixture only; mastery does not require public internet. Explain the hypothetical request/response path, then prove parsing/validation locally.

12. Small version alone

Write a function that accepts either local JSON path or URL mode through config. Both paths must feed the same validate_metrics() function.

13. Validate

Status/path source recorded; exactly four region metrics; required keys present; failed retrieval cannot reach publish.

14. Teach it back

Explain the difference between "HTTP request succeeded" and "the data is valid".

15. Protected review + changed retry

Attempt protection

After review, point the local mode at service_metrics.json or another provided JSON and adapt only the validation contract, not retrieval plumbing.

16. Evidence / mastery + next route

Ready when

Save offline validation, local HTTP evidence if available, timeout/status code logic, one failure route. PASS -> L07.

B02-L07 - Error handling, failure classification and containment

Bridge route

What you already know as an analyst: fix an error interactively and rerun a cell.

What changes in engineering: the code must expose what failed, with context, and stop or quarantine according to policy.

Why engineers care: nobody may be watching the run live.

Small worked example: sales_bad_numeric contains order 1006 with unit_price="oops".

Real engineering version: C3 adds transient retry policy, orchestration failures and incident recovery.

1. Why it matters

A pipeline that hides an error can be more dangerous than one that crashes. The goal is not "no exceptions"; it is to classify failure, add context, contain damage and block unsafe publish.

2. What you are learning

Distinguish missing-input, schema, parse, quality and network failures; catch only what you can contextualize/recover from; raise meaningful errors; quarantine or block deliberately.

3. Prerequisite

L06 mastered; basic try/except syntax may be familiar but no engineering error strategy is assumed.

4. Picture it in your head

Think of circuit breakers. A small problem in one appliance should not burn the house. Some failures can be isolated; critical ones cut power before damage spreads.

5. New words before we use them

exception - a runtime signal that normal execution cannot continue as written

classification - naming the kind of failure so the correct recovery is chosen

containment - preventing a bad condition from contaminating later output

quarantine - separating invalid records for review instead of silently mixing/dropping them

6. Watch / visual source

Python Errors and Exceptions is authority. Optional Corey try/except: roughly 00:21-03:00 covers missing-file and specific-vs-general exceptions.

[Python 3.14.7 Errors and Exceptions](#) | [Corey try/except visual](#)

7. Follow with me once

Run the bad-numeric fixture through a strict parser and save the failure. Run missing-column fixture through require_columns(). Classify each as parse vs schema failure and prove final publish is not called.

```
def parse_numeric(series, field):
    parsed = pd.to_numeric(series, errors="coerce")
    bad = parsed.isna() & series.notna()
    if bad.any():
        raise ValueError(f"Malformed {field}: rows={bad.sum()}")
    return parsed
```

8. What you should see

The evidence identifies one malformed numeric at order 1006 and missing customer_id in the second fixture. Neither failure creates a trusted final output.

B02-L07 - Practice to mastery

9. Why it worked

Specific validation turns vague downstream wrong totals into early, contextual failures; failure boundary sits before irreversible/published output.

10. Common beginner mistakes

- except Exception: pass.
- Coercing bad numeric to 0 without policy/evidence.
- Logging ERROR and continuing to publish anyway.
- Deleting a malformed row and calling it "cleaning" with no quarantine/control.

11. If stuck

Recovery ladder

Test one failure at a time. First assert required columns. Then numeric parsing. Save the exact exception before writing recovery logic.

12. Small version alone

Create a copied input with one invalid discount_pct > 1.0. Classify it as business-quality failure (not parse failure) and block publish.

13. Validate

Failure type, affected row/key, policy (block/quarantine), output-withheld proof and clean-input pass are all recorded.

14. Teach it back

Explain why "caught the exception" is not automatically a successful recovery.

15. Protected review + changed retry

Attempt protection

Open review only after attempt; changed retry uses a different malformed field and requires the same containment behavior.

16. Evidence / mastery + next route

Ready when

Save exception messages, failure classification table, output-withheld evidence, clean rerun. PASS -> L08.

B02-L08 - Logging, run evidence and validation gates

Bridge route

What you already know as an analyst: `print(df.shape)` and inspect manually.

What changes in engineering: save timestamped run events plus structured checks that can be reviewed later.

Why engineers care: unattended jobs need reconstructable evidence and objective pass/fail controls.

Small worked example: `customer_key_unique` FAIL must block publish even if Python raised no unexpected exception.

Real engineering version: C3-08 expands this into observability, incidents, lineage and DataOps.

1. Why it matters

When a pipeline runs without you watching, you need evidence of what happened and whether the data passed rules. Logs describe events; validation results describe trust.

2. What you are learning

Use meaningful log levels, create machine-readable validation results, distinguish operational success from data-quality success, and gate publish on critical checks.

3. Prerequisite

L07 mastered.

4. Picture it in your head

A flight has two records: the black-box event log and the safety checklist. "The engines ran" does not prove "the aircraft was safe to depart." Pipeline logs and validation results play different roles.

5. New words before we use them

log level - severity label such as INFO/WARNING/ERROR

observability evidence - recorded signals that help reconstruct what a run did

validation result - structured check with rule/status/observed value

publish gate - decision point that prevents final output when critical checks fail

6. Watch / visual source

Python 3.14.7 Logging HOWTO is authority. Optional Corey Logging Basics: full 14:25; focus on replacing print, log levels, file logging and format.

[Python 3.14.7 Logging HOWTO](#) | [Corey Logging Basics](#)

7. Follow with me once

Create `run.log` and `validation_results.json` for the practice batch. Record discovery count, raw rows, parse result, master uniqueness and reconciliation. Make `customer uniqueness` a critical check.

```
checks = [{
    "check": "customer_key_unique",
    "status": "PASS" if customers["customer_id"].is_unique else "FAIL",
    "observed": int(customers["customer_id"].nunique()),
    "rule": "customer_id must be unique"
}]
logger.info("validation_complete checks=%s", len(checks))
```

8. What you should see

`run.log` tells chronological events; `validation_results.json` exposes `customer_key_unique=FAIL` on the supplied ambiguous master; final publish is withheld.

B02-L08 - Practice to mastery

9. Why it worked

The validation schema is machine-readable and independent from human prose logs. A critical FAIL has an enforced consequence.

10. Common beginner mistakes

- Everything logged as INFO.
- Passwords/tokens or private data written to logs.
- A JSON check says FAIL but code still publishes.
- Using logs instead of an independent numeric reconciliation.

11. If stuck

Recovery ladder

Start with three events: `run_started`, `inputs_discovered`, `validation_complete`. Then create only two checks: `expected_file_count` and `customer_key_unique`.

12. Small version alone

Add a `net_sales` reconciliation check with tolerance 0.01 and prove it passes on the three good monthly files.

13. Validate

Saved log exists; validation JSON parses; each check has rule/status/observed; at least one controlled FAIL blocks publish.

14. Teach it back

Explain why "no ERROR log" does not prove a dataset is trustworthy.

15. Protected review + changed retry

Attempt protection

After review, add one fresh schema check and inject its failure. Do not change logging setup to solve the data problem.

16. Evidence / mastery + next route

Ready when

Save `run.log`, `validation_results.json`, publish-gate decision, changed-failure retry. PASS -> L09.

B02-L09 - Recurring batch execution and safe rerun reasoning

Bridge route

What you already know as an analyst: rerun a notebook/report manually.

What changes in engineering: define what repeat execution is allowed to change and prove duplicate business rows are not accumulated.

Why engineers care: schedulers will eventually rerun jobs after retries/failures.

Small worked example: write a complete output snapshot to a controlled path rather than append unchanged monthly rows every run.

Real engineering version: C3 teaches idempotence/runbooks, checkpoint state, replay and backfills.

1. Why it matters

A recurring job will run again. If the second run appends the same business rows or depends on leftovers from the first, automation creates corruption instead of saving time.

2. What you are learning

Coordinate discovery -> load -> transform -> validate -> publish as one rerunnable batch, keep raw inputs immutable, use controlled output replacement/versioning, and record a run manifest.

3. Prerequisite

L01-L08 mastered; C1 reproducibility basics familiar.

4. Picture it in your head

Imagine a photocopier with a RUN button. Pressing RUN twice should produce another copy of the intended report, not secretly staple a duplicate set of pages into the business ledger.

5. New words before we use them

rerun - execute the same workflow again

idempotence - a property where repeating an operation does not create additional unintended effect

run manifest - record of versions, inputs, counts, checks and outputs for one run

immutable input - source data kept unchanged by processing

6. Watch / visual source

Book-led. pathlib/logging docs support implementation. The old venv video is removed from B02 active study because C3-00 teaches environment/dependency reproducibility from zero.

[Python 3.14.7 pathlib](#) | [Python 3.14.7 Logging HOWTO](#)

7. Follow with me once

Run a controlled good-path batch using a master copy you explicitly make unique only for demonstration (recording that the source master remains invalid). Produce CSV/JSON evidence. Run it twice without changing inputs.

```
def run_batch(config):
    files = discover_files(config)
    sales = load_many(files)
    clean = transform(sales)
    checks = validate_all(clean)
    if any(c["critical"] and c["status"] == "FAIL" for c in checks):
        raise RuntimeError("Critical validation failed; publish blocked")
    return publish_snapshot(clean, checks, config)
```

8. What you should see

Business-row count and output hash/content are stable across unchanged runs; source files remain unchanged; a new run manifest records each run without duplicating business rows.

B02-L09 - Practice to mastery

9. Why it worked

The pipeline separates append-only run evidence from replace/versioned business output. Repetition is planned rather than accidental.

10. Common beginner mistakes

- Append mode for a full-snapshot output.
- Deleting raw inputs after success.
- Second run succeeds only because first run created hidden temp state.
- Calling a workflow idempotent without defining the side effect being protected.

11. If stuck

Recovery ladder

Define the side effect first: final business output. Make it a fresh snapshot file. Prove row count before adding any scheduling idea.

12. Small version alone

Change only output directory and rerun from clean process. The data result must match while run path/manifest reflects the change.

13. Validate

Two unchanged runs -> same business rows/totals; raw file hashes unchanged; manifest lists exact selected inputs and runtime versions.

14. Teach it back

Explain idempotence in your own example and name one operation that is not automatically idempotent.

15. Protected review + changed retry

Attempt protection

After review, inject a failed validation between transform and publish. Prove the previous good output is not overwritten by the failed run.

16. Evidence / mastery + next route

Ready when

Save two run manifests, output row/total/hash comparison, source hashes, failed-run output-protection proof. PASS -> L10.

B02-L10 - Build and defend a small reproducible batch pipeline

Bridge route

What you already know as an analyst: integrated notebook produces an analysis/report.

What changes in engineering: the artifact is a rerunnable system with contracts, controls, evidence and safe failure.

Why engineers care: C3 assumes you can reason about data moving through stages before it adds real ingestion, dbt, orchestration and platform tools.

Small worked example: three sales files -> validated rows -> safe master join -> regional output + run evidence.

Real engineering version: C3-01 begins source->landing->staging, APIs, databases, incremental state, CDC and backfills.

1. Why it matters

Bridge mastery must prove you can combine the ideas, not just answer isolated questions. This lesson is the rehearsal for the B02 Mini-Lab and protected Gate.

2. What you are learning

Build a small end-to-end local batch pipeline with explicit contract, deterministic discovery, parsing, merge safety, failure containment, logging, validation, safe publish, rerun evidence and handoff.

3. Prerequisite

B02-L01 through L09 mastered.

4. Picture it in your head

Think of a transparent pipe with inspection windows. Data enters through named inputs; every stage has a check; a red gate stops unsafe data; the final output comes with a receipt showing exactly how it was produced.

5. New words before we use them

pipeline - connected stages that move/transform data under explicit rules

handoff - evidence/instructions enabling another person to run and diagnose the work

critical failure - defect that makes publishing unsafe

defense - concise explanation of architecture, controls, failure behavior and limitations

6. Watch / visual source

No video required. Use the lesson as teacher; revisit only the exact prior source for the skill you cannot reproduce.

[Python 3.14.7 Modules](#) | [pandas 3.0.5 User Guide](#)

7. Follow with me once

Build the pipeline using the provided practice data. The supplied duplicate C013 is a required failure: first prove the run blocks. Then use a clearly documented, separate demonstration master (not silent repair of source) to complete the good-path rehearsal.

```
DISCOVER -> VALIDATE FILE SET -> LOAD -> PARSE/TYPES
-> QUALITY CHECKS -> MASTER-KEY CHECK -> MERGE(validate=...)
-> BUSINESS RECONCILIATION -> PUBLISH GATE
-> OUTPUT + validation_results.json + run.log + run_manifest.json
```

8. What you should see

First run stops safely on duplicate master. Good-path rehearsal produces validated output and evidence. Repeating unchanged good inputs does not duplicate business rows.

B02-L10 - Practice to mastery

9. Why it worked

Every critical assumption is either declared or checked. Failure evidence is retained, and publish is downstream of validation.

10. Common beginner mistakes

- Editing customers.csv until the test passes.
- Publishing before validation because "the rest of the code needs an output".
- A manifest with filenames but no counts/check status/versions.
- Calling the capstone complete without a failure/recovery demonstration.

11. If stuck

Recovery ladder

Build vertical slices: discovery+count first; then parse+reconcile; then master-key block; then good-path merge; then logs/checks; publish last.

12. Small version alone

Rebuild the same architecture on a fresh two-file mini-batch and a clean, separately created master. Keep function names/contracts but do not copy final outputs.

13. Validate

Prove selected files, 96-row source control on full practice batch, ■370,375 source net total, duplicate-key block, good-path row preservation, validations, safe second run, manifest and no raw edits.

14. Teach it back

Give a 2-minute defense: input/output grain, stage boundaries, most dangerous failure, control that catches it, publish rule, rerun behavior, C3 next step.

15. Protected review + changed retry

Attempt protection

Protected Review L10 only after attempt. Changed retry uses a fresh mini-batch; then complete B02 Mini-Lab from a clean copy.

16. Evidence / mastery + next route

Ready when

Save project files, run evidence, validation JSON, two-run proof, failure/recovery proof and defense notes. PASS -> B02 Mini-Lab, then Gate A.

B02 Quick Reference & Next Route

Question	Engineering answer
Input set	Which files/records were selected, and by what rule?
Grain	What does one row/object represent before and after each major stage?
Schema/type	Which columns/types are required, and what happens if they are wrong?
Join	Is the master key unique? What cardinality is declared? Any unmatched keys?
Quality	Which failures BLOCK and which can be WARN/quarantine?
Evidence	What did the run do? What checks proved trust?
Rerun	What happens if the same inputs run twice?
Handoff	Can another learner reproduce the run without hidden notebook state?

Assessment route

Complete B02 Mini-Lab independently. Then Gate A is the only normal full Gate. Failed Gate A -> targeted repair; Gate B opens only by explicit full-retest assignment. Failed/unstable B -> second targeted repair; Gate C opens only by explicit assignment. Reviews unlock only after the matching genuine attempt is saved.